Backup and Disaster Recovery for Containers Running on Kubernetes

ARCHIVED 12 July 2024 - ID G00810147 - 22 min read

By: Fintan Quinn, Tony Iams

Initiatives: Delivery of Functional Responsibilities

I&O technical professionals must adopt new tools and processes for backup and disaster recovery with containers and Kubernetes. This research highlights key areas that require attention for containerized applications running on Kubernetes to ensure resilience.

Overview

Key Findings

- Backup and recovery in Kubernetes (K8s) remains an evolving domain with ongoing discussions regarding the optimal approach. While vendor offerings have seen enhancements, they continue to contend with initiatives driven by the open-source community.
- Kubernetes manages a diverse array of stateful workloads, including relational and NoSQL databases, software-as-a-service (SaaS) data services, and virtual machines (VMs). This diversity creates a demand for backup and recovery mechanisms capable of orchestrating comprehensive recovery processes, encompassing all application components and their associated data.
- DevOps practices using infrastructure as code (IaC) and continuous integration/continuous deployment (CI/CD) for application deployment and redeployment are commonly seen as methods of recovery. These approaches do not consider application data stored as persistent volumes (PVs) that need additional protection measures to ensure recovery.
- Cloud-native and traditional infrastructure engineers have different views on K8s backup and recovery. Cloud-native engineers prioritize redeployment methods via CI/CD workflows, while traditional approaches rely on backup and recovery tools for Kubernetes application and data protection.

This research note is restricted to the personal use of .

Recommendations

- Assess all Kubernetes clusters to identify both the specific workloads and their recovery requirements. Implement a comprehensive blend of backup and recovery methods to ensure that applications receive necessary protection from failures, misconfigurations and security threats.
- Create a shortlist of backup vendors offering Kubernetes integration, automation, policy-based operations, multiplatform support, and features for reporting and alerting.
- Assess the importance of all persistent volume claims (PVCs) in your production clusters. Implement backup mechanisms that ensure recovery expectations can/will be fulfilled.
- Integrate Kubernetes backup processes into DevOps workflows that are transparent to the development process. This will automate the backup of application configuration (in tandem with CI/CD), including stateful data, and provide the ability to replicate applications into dev and test environments, while also supporting cluster migration and upgrades.
- Define clear responsibility for Day 2 tasks in Kubernetes operations. Ensure backup and recovery is delegated to teams that will deliver recovery objectives in line with business requirements.

Strategic Planning Assumption

Gartner predicts that by 2026, 15% of on-premises production workloads will run in containers, up from less than 5% in 2022.

Analysis

Kubernetes is now the de facto standard for container orchestration and has become the foundational platform for many applications. Gartner sees enterprises deploy containers and Kubernetes at increasing scale as they seek to apply cloud-native computing to modernize infrastructure and applications across their organizations.The rising utilization of Kubernetes for critical workloads has introduced new challenges that infrastructure and operations (I&O) technical professionals must address, notably in the realm of backup and recovery. Operating container infrastructure based on Kubernetes in an enterprise environment requires the ability to back up and recover the state of stateful applications running on a Kubernetes cluster and, in some cases, the associated data.

This research note is restricted to the personal use of .

Backup and recovery for containerized applications running on Kubernetes requires a distinct strategy and implementation compared to traditional monolithic applications utilizing server/VM architecture. This strategy must account for the following:

- The risks, and why Kubernetes requires backup and disaster recovery planning (DRP)
- The available options for both technical solutions and the operational processes to use them
- DevOps and automated redeployment approaches to recovery
- The evolution of stateful applications in a container environment
- The different personas responsible for Kubernetes backup and recovery

The rest of this analysis section explores these areas and highlights key considerations.

Why Does Kubernetes Require Backup and DRP Capabilities?

Kubernetes clusters are complex systems that include nodes, pods, services and persistent volumes. These components are spread across different hosts and can be susceptible to failures or errors, whether caused by hardware or software issues, human error, or even cyberattacks. A component's failure can lead to data loss, downtime or reduced performance.

See How to Self-Assess Your Kubernetes Maturity, 1.3 Practice: Continuity Strategy, for additional information on Kubernetes resilience strategies.

The Kubernetes architecture does not specify how backup or DR should be implemented with clusters. Therefore, I&O technical professionals will have to arrive at their own solution, based on the needs of applications deployed on Kubernetes clusters.

To ensure a complete backup and recovery strategy for Kubernetes, it is essential to consider three key areas, as shown in Figure 1:

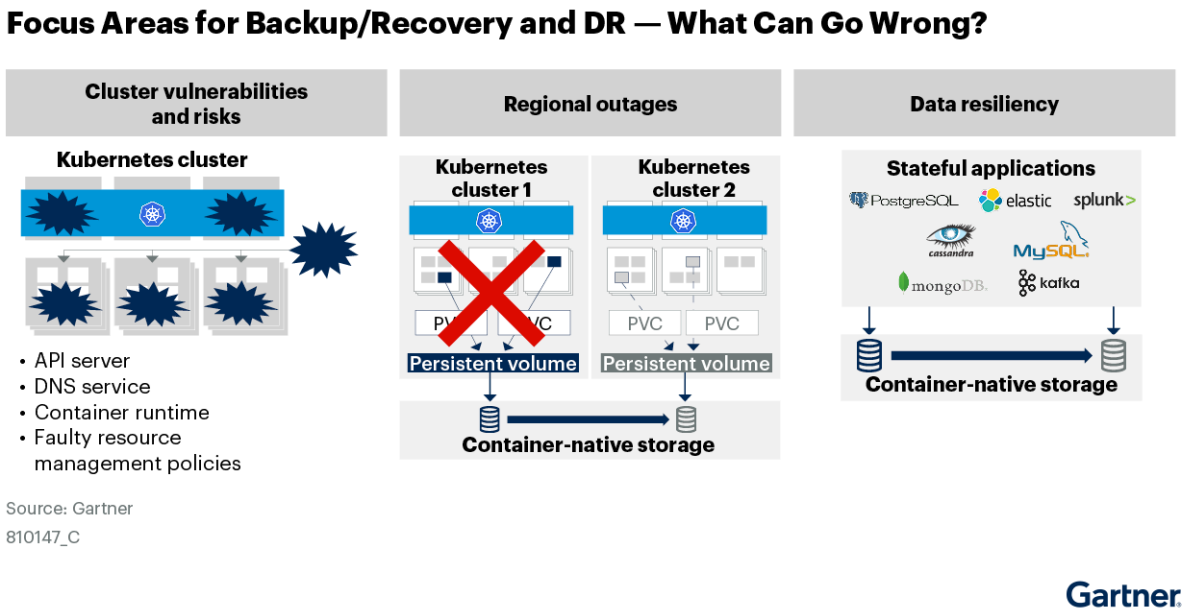
This research note is restricted to the personal use of .

- Cluster vulnerabilities and risks (in region) with the potential for catastrophic failures and security breaches that can affect all applications running on a cluster

- Regional outages (disaster recovery [DR])

- Data resiliency and protection from cyberthreats such as ransomware (stateful applications)

Figure 1: Focus Areas for Backup/Recovery and DR



Global Cluster Risks

Potential risks to the operation of an entire cluster include:

- API server: The API server is the central entry point through which all users and components access a Kubernetes cluster, and it can become a bottleneck. The growing use of operators and custom controllers using custom resource definitions (CRDs) highlights the strain that can be put on the API server, as well as etcd.

- DNS service: Kubernetes fundamentally depends on DNS for service discovery. Should the cluster's DNS service become overloaded or misconfigured, it will likely affect the performance and functionality of every application service that runs on the cluster.

This research note is restricted to the personal use of .

- Container runtime: By default, the container runtime on each worker node is shared by all pods running on that node. If the runtime becomes unresponsive, it will impede the performance of all pods on the node.
- Faulty resource management policies: As application services deployed on Kubernetes become more complex — and pods increase their resource consumption with "sidecar" containers for such tasks as logging and monitoring — the risk of catastrophic misconfigurations increases. Guidance on optimizing Kubernetes performance is available in the research note, Best Practices to Optimize Kubernetes Performance.

These issues can lead to a domino effect, causing the entire cluster to become unresponsive. Moreover, as with any complex software system that is regularly upgraded with new code releases and security patches, it's possible that a routine software update can cause serious failure conditions that require restoring a Kubernetes cluster to an earlier state.

Regional Outages

Network latency requirements make it difficult to stretch the K8s control plane over long distances. Hence, we need a DR strategy for containerized applications running on K8s in case of infrastructure failures affecting the entire site or region. The following deployment types must be considered for such a strategy:

- **Self-managed clusters:** On-premises clusters like Red Hat OpenShift, SUSE Rancher or VMware Tanzu with production applications necessitate maintaining separate clusters across geographically separated data centers. This ensures the protection of containerized application services against data center failures. These implementations must align with an organizationwide strategy for DR.
- **Public cloud deployments:** When creating and managing applications in the cloud, there's always a risk of disruption from outages and disasters. When utilizing public cloud Kubernetes services such as Microsoft Azure Managed Kubernetes Service (AKS), Amazon Elastic Container Service for Kubernetes (Amazon EKS) or Google Kubernetes Engine (GKE), it's important to differentiate between high availability (HA) and disaster recovery. While the service provider ensures HA (through the use of availability zones [AZs]), it's the responsibility of the customer to manage recovery in an alternate region.

This research note is restricted to the personal use of .

- **Managed Kubernetes platforms:** Platforms like Giant Swarm, Platform9 and Kubermatic prioritize resilience and availability utilizing AZs. While these platforms offer numerous advantages for developers, such as speed of deployment and agility, it's crucial for production applications to have recovery strategies aligned with business expectations. These platforms vary in how backup is provided and whether regional DR is available or not. The same level of control is not available as it would be with public cloud deployments.

Data Resiliency and Protection

One common misconception about Kubernetes is the idea that applications running on the platform must be designed to be stateless. In reality, most applications and projects running on a Kubernetes cluster will eventually require storing or accessing data.

Kubernetes stateful applications/workloads use PVs to store data. A PV has an independent life cycle from the pod that is consuming it, so that data can be preserved on the underlying storage system even if the pod goes away. However:

- What if the underlying volume on the storage system gets corrupted for some reason, for example, a cyberattack?
- What if the underlying storage system is stricken by a disaster?

When that happens, even data stored on the PVs will be gone. To prevent data loss, we need to find a way to protect the data stored in the PVs used by the Kubernetes stateful applications/workloads. This will be covered in more detail in a later section dedicated to the topic of state within Kubernetes.

Kubernetes is a unique computing platform, which differs from traditional ones. Like any platform, if not properly designed and operated, it can fall prey to common data loss risks inherent in both private and public cloud environments.

However, Kubernetes provides a consistent operating environment across platforms, and with the appropriate tools, applications and their data can transition between various platforms, including self-managed and public cloud environments. This flexibility opens up opportunities to innovate and devise creative recovery strategies for applications, especially when considering disaster recovery plans.

Available Options and Limitations

This research note is restricted to the personal use of .

In this section, we explore the backup and recovery options available for Kubernetes. This includes a combination of community offerings, native platform capabilities and third-party backup options, which are summarized in Table 1. Evaluate these options to determine existing operation measures and identify potential protection gaps.

Table 1: Kubernetes Backup Options

(Enlarged table in Appendix)

	Methods	Benefits	Cautions/limitations	Used by
Native/inbuilt	Etcd backup Volume snapshots Custom-made scripts	Available without investment Backups available for portability and redeployment	Scripting skills required Custom script-based approaches Lacking monitoring and reporting Lacking standards Not applicable to all use cases	Kubernetes administrators Application developers
Community	Velero Kanister	Large community projects Cost-effective option	Community-based best-effort support Lack of management features (CLI-driven) Lacking advanced recovery options	Kubernetes administrators Application developers
Platform option	AKS (Azure Backup) GKE (Backup for GKE) Tanzu Mission Control OpenShift API for Data Protection (OADP)	Cost-effective in comparison to third-party option Speed of implementation	Limited feature set Bound to the platform it protects	
Third-party backup	Veeam Kasten Trilio Portworx NetAppAstra Control	Management UI with rich user experience Support provided with SLA	Capabilities vary between vendors Supported Kubernetes platforms vary between providers	Infrastructure practitioners Backup operators

Source: Gartner (July 2024)

Native/Inbuilt Cluster Options

Kubernetes provides built-in backup and recovery options through API resources and essential maintenance tasks. Understanding the built-in backup options within Kubernetes is crucial to determine their applicability and limitations. Inbuilt options include etcd backup, volume snapshots and scripted backups.

This research note is restricted to the personal use of .

Etcd backup: All persisted state for a Kubernetes cluster is registered in the control plane's etcd key-value store — which is, by design, the only stateful piece of a Kubernetes environment. Kubernetes is a distributed system, and etcd provides a distributed key-value store where Kubernetes control plane can read and write data. Kubernetes administration best practices outline periodically backing up the etcd cluster data.

Etcd backups are achieved by generating snapshots of etcd through scheduled tasks and custom scripts. These snapshots can subsequently be transferred from the cluster to an external storage location. While this method is considered a best practice, it's important to address certain misconceptions about etcd backup, as outlined in Table 2.

Table 2: Etcd Backup Use Cases and Drawbacks

Etcd backup use cases	Etcd backup drawbacks
▪ Forensics and troubleshooting ▪ Legal and compliance ▪ Preupgrade	▪ No partial recovery — all or nothing ▪ No access to etcd backup on managed platforms such as AKS, EKS or GKS ▪ Configuration drift — inconsistency between etcd database and Kubernetes cluster ▪ Requires custom scheduling

Source: Gartner (July 2024)

Simply backing up etcd is not enough to protect applications and associated data, and MUST be supplemented with one or more backup and recovery approaches.

This research note is restricted to the personal use of .

Volume snapshots: Persistent volumes presented to pods can be protected by utilizing the volume snapshot API object. Volume snapshots provide Kubernetes users with a standardized way to copy the contents of a PV at a particular point in time without creating an entirely new volume.

Scripted backups: With the appropriate kubectl commands, Kubernetes resource definitions can be stored as YAML files in locations like Git repositories. These methods are customized using scripts provided/developed by the cloud-native community and adapted for specific use cases.

Community

- **Velero:** An open-source tool that assists in automating backup and restore of Kubernetes clusters. Velero can either back up all objects in the cluster or be more granular by filtering objects by namespace or label. Velero is a popular solution among the cloud-native and Kubernetes communities. It is often Kubernetes admins' first option because it is easily accessible without licensing costs and does not lock users into any particular system. A strong and active community support, as well as trust in open source, are selling points for Velero within the Kubernetes community.

- **Kanister:** This open-source project and community since 2017 is an extensible framework for application-level data management on Kubernetes. Kanister can capture data management tasks in application-specific blueprints, facilitating easy sharing and extension. These blueprints can create application-consistent backups for persistent workloads, including databases like Cassandra, PostgreSQL, MySQL and Elasticsearch. Third-party backup vendors such as Kasten by Veeam use Kanister integration to provide better application consistency for backups of stateful workloads.

Platform Options

Certain Kubernetes platforms incorporate backup and recovery as part of the offering:

- Google provides this as part of the Google Kubernetes Engine, providing the ability to restore each backup to a cluster in the same region or, alternatively, to a cluster in a different region.

- Mission Control in VMware Tanzu includes backup functionality, making the backup and DR process more streamlined and easier to implement.

This research note is restricted to the personal use of .

- AKS now has AKS backup as part of Azure backup to back up AKS workloads and persistent volumes.
- Red Hat OpenShift offers the OpenShift API Data Protection (OADP) interface, which is based on the Velero APIs for backup, restore, schedule, backupstorageLocation and VolumeSnapshotLocation.
- Rancher includes the “rancher-backup” operator, which allows backups of Kubernetes resources to a variety of persistent storage locations.

Some of the following examples leverage the Velero workflow to provide both front-end and protection capabilities.

Third-Party Tools

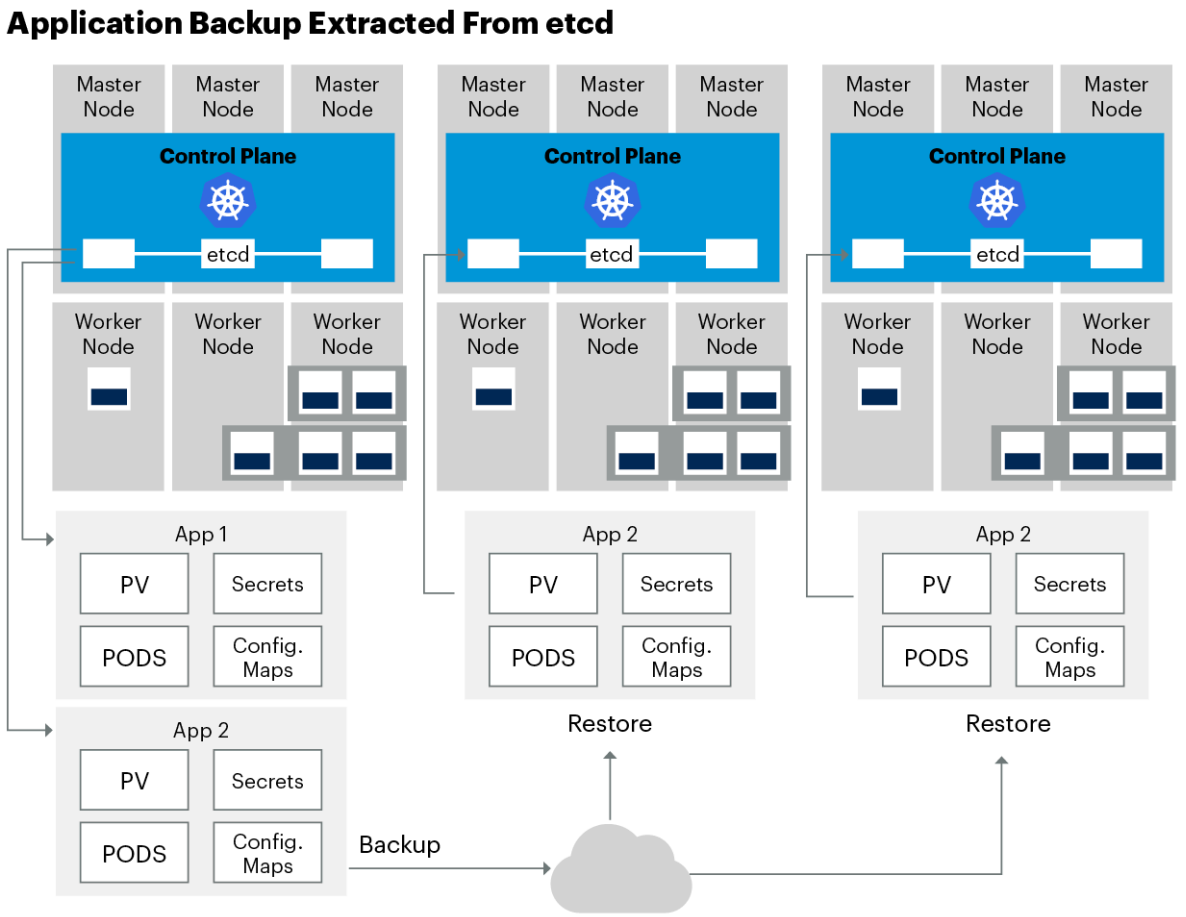
Commercial products from Kubernetes specialized vendors, such as Trilio, CloudCasa and Kasten, and traditional backup vendors with enhanced support for containers, such as Commvault and Rubrik, now aim to address the dynamic cloud-native requirements. Kubernetes backup platforms now focus on becoming ingrained in the Kubernetes cluster. Deployed using custom resource definitions (CRDs) into dedicated namespaces within the cluster, Kubernetes-aware backup software/platforms aim to fully understand the objects within a Kubernetes cluster and how they make up an application (i.e., how an application is composed).

Outlined in Figure 2, the methodology aims to:

- Capture metadata from the control plane/etcd, including secrets, config maps, service accounts, pods and other Kubernetes objects associated with an application.
- Define applications as YAML/JSON files and store them in a secure location for granular, namespace-based restoration across multiple platforms.
- Achieve dynamic application discovery through access to control plane metadata.
- Offer more backup and restore options, providing greater granularity and flexibility for migrating applications.

This research note is restricted to the personal use of .

Figure 2: K8s Backup



Source: Gartner
746842_C

Gartner

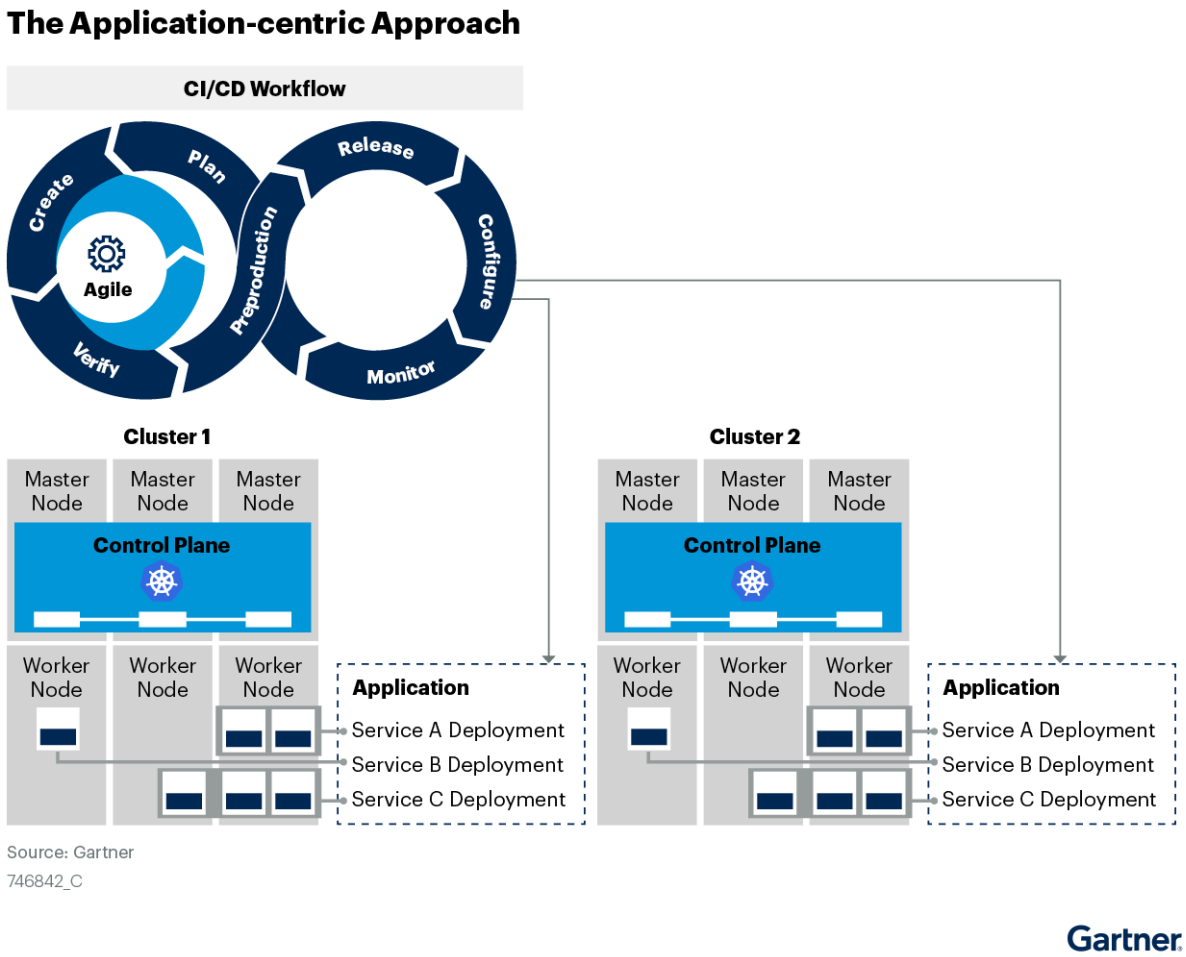
DevOps Approaches — Recover via Redeployment

DevOps principles, alongside Kubernetes, have paved the way for recovery through redeployment. The premise here is that Kubernetes deployments are defined in code so they can be recreated or reinitialized on demand if they fail, and then repopulated with applications using a CI/CD workflow.

This research note is restricted to the personal use of .

Figure 3 provides an overview of application deployment via a CI/CD workflow to the primary cluster (Cluster 1), while the workflow can also deploy applications to a separate cluster (Cluster 2), which can be located on a separate infrastructure or cloud platform. This can be defined as an application-centric approach where applications are defined in a GitOps manner, allowing you to utilize tools like ArgoCD/Flux CD to deploy them across multiple clusters seamlessly. See How to Self-Assess Your Kubernetes Maturity, 2.1 Practice: Cluster Provisioning for additional information and Is Using GitOps-Style Automation With Kubernetes Right for Me? for further information on GitOps practices.

Figure 3: DevOps Approach



The model of having an application and resource configuration defined as code is, of course, beneficial to everyone. In terms of operationalizing such an approach, the following factors must be considered:

- This approach was never intended for stateful applications, as the proliferation of persistent data is now a factor in the majority of containerized workloads.

This research note is restricted to the personal use of .

- Not all organizations will have the skills required to implement the CI/CD and DevOps methodologies to a mature level and integrate this with a resiliency strategy for true automation. (This will be discussed further in the personas section.)
- Application-centric backup and recovery methods based on DevOps redeployment often lack governance and oversight in comparison to backup and recovery of traditional workloads.

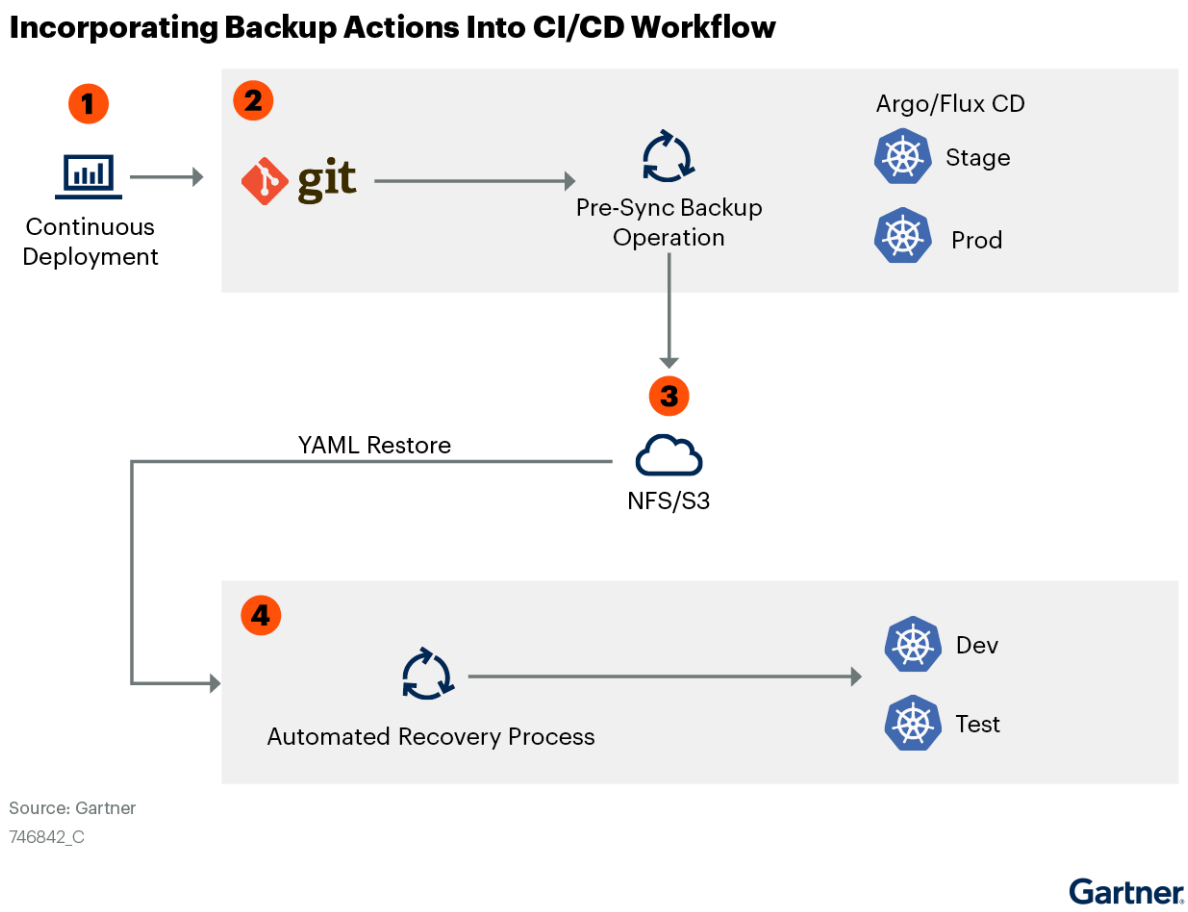
It is clear that recovering applications via deployment in this automated fashion restores only Kubernetes objects and configuration. However, there is an opportunity with the right tooling in the form of third-party backup to combine these DevOps approaches with complete application backup. Certain backup tools have been designed to be specifically Kubernetes-native such as Trilio and Kasten with the capability to embed themselves into the CI/CD pipeline so that backup and recovery become part of the workflow.

As shown in Figure 4, when a change is pushed through the pipeline, the following sequence is initiated:

1. Scheduled pipeline or schema change is invoked.
2. A CD process like Argo or Flux CD looks to merge, which invokes a presync backup job.
3. The entire application is captured as a YAML file stored in external NFS/S3.
4. Further automation can be leveraged to import backup files to a target cluster for potential test and development use cases.

This research note is restricted to the personal use of .

Figure 4: Incorporating Backup Actions Into CI/CD Workflow



We Have to Talk About State

Containers have traditionally been favored for running stateless web and batch applications. Although this statement may have been accurate during the project's early stages, the platform's support for storage and stateful applications matured quite some time ago. Presently, over 41% of container organizations are utilizing containers to host databases. [1]

> **Running stateful workloads within your Kubernetes cluster is okay, but it's crucial to adapt backup and recovery processes to include application data.**

Three misconceptions regarding backup and recovery for these workloads need to be dispelled:

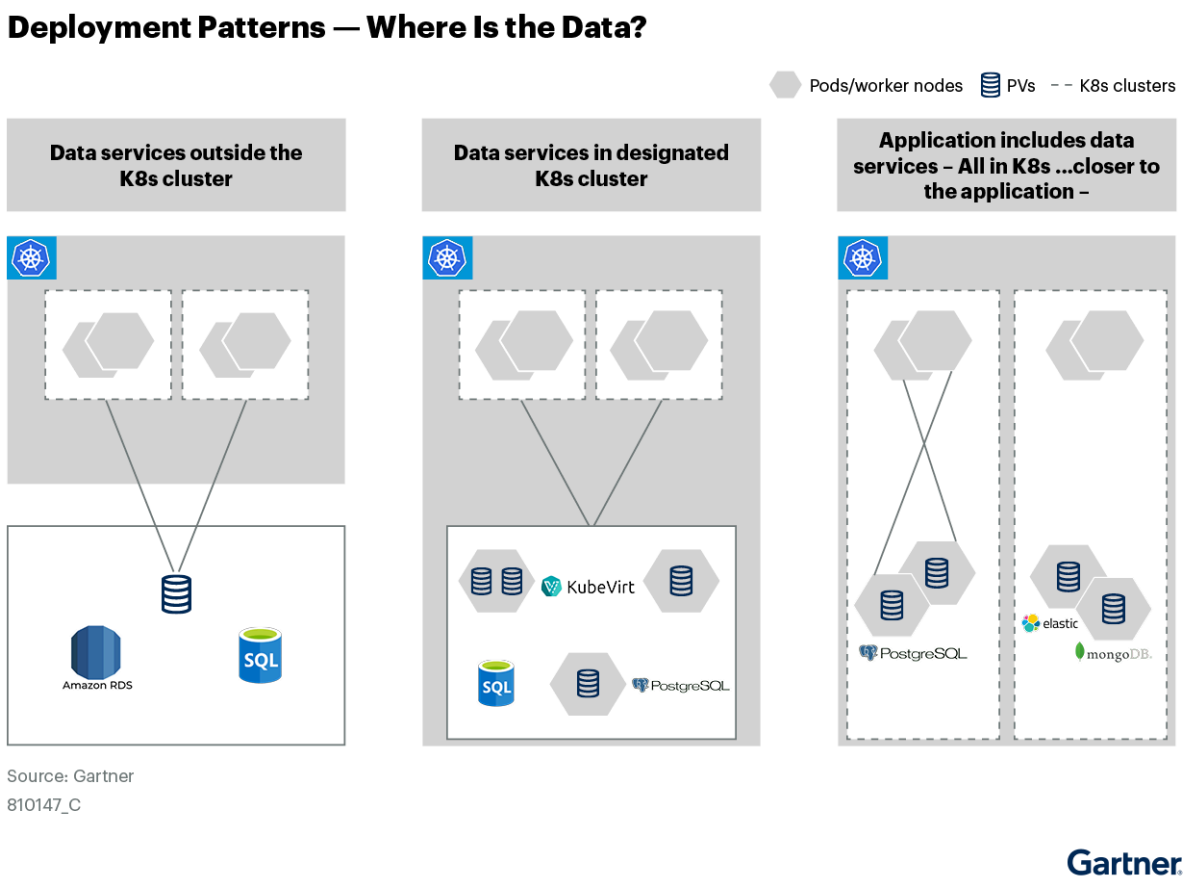
This research note is restricted to the personal use of .

- GitOps can be used to redeploy applications — persistent data is not captured in version control.
- Numerous applications believed to be stateless often have PVs defined. Upon further analysis, they may reveal Prometheus metrics or logging enabled, necessitating some degree of data protection.
- Public cloud data availability and resilience protects my applications — this won't protect against data corruption, whether accidental or malicious.

The trend for managing persistent data in containers has grown in the last few years. The data collected for the most recent Cloud Native Computing Foundation (CNCF) Survey published in December of 2023 showed nearly 60% of respondents running stateful applications in containers in production (an increase from 52% in the 2021 CNCF Survey). [2,3] Kubernetes offers the PV/PVC mechanism, enabling applications to utilize the cluster's storage infrastructure to fulfill their storage requirements. The most common standard to achieve this is to use the container storage interface (CSI). With the adoption of the CSI, the Kubernetes volume layer is now truly extensible. Using CSI, third-party storage providers can write and deploy plug-ins exposing new storage systems to Kubernetes without ever having to touch the core Kubernetes code. This gives Kubernetes users more options for storage and makes the system more secure and reliable. Three widely adopted cloud native deployment patterns involving data services are illustrated in Figure 5.

This research note is restricted to the personal use of .

Figure 5: Deployment Patterns — Where Is the Data?



Data services outside of Kubernetes (off cluster) — This pattern is commonly used for databases located outside the Kubernetes cluster. It allows developers to concentrate on enhancing applications without concerning themselves with the operational aspects of their databases. Additionally, it offers built-in high availability and automated backup capabilities for operational resilience. However, the backup approach is distinct from the complete application, posing the risk of a fragmented recovery process.

Data services in separate Kubernetes cluster (in cluster) — This pattern will aim to isolate storage from the application by using separate pods and worker nodes with dedicated storage that can be allocated to workloads in separate pods when required.

Application includes data services (in pod) — Developers may want to deploy a workload as a single unit by locating data services within the same pods and cluster as the workload. This may be scaled differently for production, but is often used for test and development or when performance may be a factor.

This research note is restricted to the personal use of .

In many cases, the choice of pattern is shaped by the available skill sets and the personas involved, a topic we will detail further in the upcoming section of the analysis.

We have to consider all workloads managed by Kubernetes and their protection requirements. Kubernetes has evolved beyond container orchestration for quite some time now. Examples of this evolution include virtual machines such as OpenShift Virtualization based on KubeVirt and the VMware VM service.

We need to fully understand our applications and where the data is stored. Table 3 provides more detail on these deployment patterns and their associated data services and use cases.

Table 3: Deployment Patterns and Their Use Cases, Benefits and Drawbacks

Deployment pattern	Use cases	Benefits	Drawbacks
Data services outside of Kubernetes	Data-managed services from infrastructure providers	Delegated management Integrated resilience features	Cost Potential for lock-in
Data services in separate Kubernetes cluster	Distributed databases Isolated and dedicated storage cluster Provides central storage for all applications	Isolates storage from application	Additional management overhead Potential bottleneck for other services if not well designed
Application includes data services	Test and dev Latency-sensitive applications	Close to application providing fast performance Fast provisioning	Less resilient Single point of failure

Source: Gartner (July 2024)

Personas and Approaches

This research note is restricted to the personal use of .

Organizations vary in their maturity levels regarding DevOps and cloud-native approaches utilizing Kubernetes for container and workload management. These differences in maturity often blur organizational lines of responsibility from a backup and DR perspective.

Traditionally, backup responsibilities rested with a central team, overseeing job settings, schedules and retention. Nowadays, with a blend of development teams familiar with Kubernetes architecture but lacking backup and DR expertise, and infrastructure/backup administrators needing to enhance their understanding of cloud-native architectures, the landscape has evolved.

Understanding these different personas is crucial, as each will have different perspectives on backup and recovery.

Application Dev Teams

Ultimately, these teams will adopt recovery through redeployment, a philosophy wherein Dev teams take on the responsibility for backing up and recovering applications. Applications teams practicing DevOps will approach backup and DRP with the following characteristics/attributes:

- Developers define application components and infrastructure requirements as code or configuration files
- Application recovery hinges on CI/CD workflow
- CLI-driven processes

Infrastructure is managed as code and applications are deployed using automated processes.

Infrastructure Teams

These teams will utilize conventional techniques for backing up and recovering Kubernetes workloads. As infrastructure teams progress beyond initial Kubernetes adoption, they will address backup and DRP in the following manner:

- Focus on the infrastructure, extracting applications from the cluster and creating a backup in a secure location.

This research note is restricted to the personal use of .

- Leverage a backup tool that provides a dashboard and management UI.
- Use backup and resilience options available within platforms such as platform-as-a-service (PaaS) databases and underlying storage.
- Provide more granular options for backup and recovery namespace, metadata and object level.

There is an increasing array of vendors within this sector, falling into categories such as:

- Traditional backup solutions offering support for Kubernetes, such as Commvault, Rubrik and Dell EMC
- Kubernetes-native developed solutions, such as Kasten from Veeam and Trilio
- Native capability of container storage solutions, such as Netapp Astra and Portworx

Those overseeing backup and recovery responsibilities need to evaluate these options and determine their suitability. This process is further complicated by the diverse operational models being employed, for example, you build it, you run it (YBI-YRI) or platform engineering practices. Kubernetes deployments and operations differ significantly from traditional VM-based applications. Nevertheless, organizations must grasp backup and recovery solutions, potentially utilizing a variety of them for complete coverage.

Recommendations

I&O technical professionals with responsibility for backup and DR (I&O practitioners) should:

- Conduct a thorough assessment of all Kubernetes clusters to precisely determine the existing workloads deployed. Develop and implement a robust combination of backup and recovery strategies, tailored to each workload, to ensure comprehensive protection against potential failures, misconfigurations and security vulnerabilities. This proactive approach will fortify applications and safeguard critical data, enhancing resilience and minimizing operational risks.
- Develop and implement resilience strategies for the inherent risks of K8s environments. This will require an intentional collaboration and design to ensure that effective, tested and trusted backup and restore procedures and tools are in place. Adopt this practice across all deployments — whether on-premises or in a hosted K8s service.

This research note is restricted to the personal use of .

- Work with DevOps teams to unify/standardize recovery methods. Integrate Kubernetes-native backup processes into DevOps workflows that are transparent to the development process. This will automate the backup of application configuration (in tandem with CI/CD), including stateful data, and provide the ability to replicate applications into dev and test environments, while also supporting cluster migration and upgrades.
- Conduct a thorough evaluation of all persistent volumes and how these are being used by your applications. Understanding the data's locations and usage is crucial for establishing backup and recovery needs. Consider factors like application consistency, granular recovery and long-term retention (LTR) in this determination.
- Clarify the various personas involved and how they influence Day 2 activities in Kubernetes operations. Understanding the roles and responsibilities of different individuals or teams within the organization is essential for effectively managing backup and recovery processes for Kubernetes deployments. Align backup and recovery methods with business requirements rather than solely focusing on technical preferences and skills.

Conclusion

Within Kubernetes (K8s), the backup and recovery domain lacks clarity, sparking continual discussions on optimal strategies and potential "antipatterns." It is essential to distinguish between high availability and backup/recovery alongside disaster recovery, as each fulfills unique roles. Kubernetes fundamentally transforms the nature of applications, rendering them into entities unlike their predecessors. Thus, navigating the complexities of backups and recovery in this dynamic environment requires a fresh perspective and tailored solutions.

Evidence

[1] 10 Insights on Real-World Container Use, Datadog.

[2] CNCF 2023 Annual Survey, Cloud Native Computing Foundation (CNCF).

[3] 2023 CNCF Annual Survey, data.world.

Recommended by the Authors

Some documents may not be available as part of your current Gartner subscription.

How to Self-Assess Your Kubernetes Maturity

Best Practices to Optimize Kubernetes Performance

This research note is restricted to the personal use of .

Guidance Framework for Deploying Containers and Kubernetes

Is Using GitOps-Style Automation With Kubernetes Right for Me?

© 2026 Gartner, Inc. and/or its affiliates. All rights reserved. Gartner is a registered trademark of Gartner, Inc. and its affiliates. This publication may not be reproduced or distributed in any form without Gartner's prior written permission. It consists of the opinions of Gartner's Business and Technology Insights Organization, which should not be construed as statements of fact. While the information contained in this publication has been obtained from sources believed to be reliable, Gartner disclaims all warranties as to the accuracy, completeness or adequacy of such information. Although Gartner insights may address legal and financial issues, Gartner does not provide legal or investment advice and its insights should not be construed or used as such. Your access and use of this publication are governed by Gartner's Usage Policy. Gartner prides itself on its reputation for independence and objectivity. Its insights is produced independently by its Business and Technology Insights Organization without input or influence from any third party. For further information, see " Guiding Principles on Independence and Objectivity." Gartner insights may not be used as input into or for the training or development of generative artificial intelligence, machine learning, algorithms, software, or related technologies.

This research note is restricted to the personal use of .

Table 1: Kubernetes Backup Options

	Methods	Benefits	Cautions/limitations	Used by
Native/inbuilt	Etcd backup Volume snapshots Custom-made scripts	Available without investment Backups available for portability and redeployment	Scripting skills required Custom script-based approaches Lacking monitoring and reporting Lacking standards Not applicable to all use cases	Kubernetes administrators Application developers
Community	Velero Kanister	Large community projects Cost-effective option	Community-based best-effort support Lack of management features (CLI-driven) Lacking advanced recovery options	Kubernetes administrators Application developers
Platform option	AKS (Azure Backup) GKE (Backup for GKE) Tanzu Mission Control OpenShift API for Data Protection (OADP)	Cost-effective in comparison to third-party option Speed of implementation	Limited feature set Bound to the platform it protects	
Third-party backup	Veeam Kasten Trilio Portworx	Management UI with rich user experience Support provided with SLA	Capabilities vary between vendors	Infrastructure practitioners Backup operators

This research note is restricted to the personal use of .

	NetAppAstra Control	Supported Kubernetes platforms vary between providers

Source: Gartner (July 2024)

Table 2: Etcd Backup Use Cases and Drawbacks

Etcd backup use cases	Etcd backup drawbacks
■ Forensics and troubleshooting ■ Legal and compliance ■ Preupgrade	■ No partial recovery — all or nothing ■ No access to etcd backup on managed platforms such as AKS, EKS or GKS ■ Configuration drift — inconsistency between etcd database and Kubernetes cluster ■ Requires custom scheduling

Source: Gartner (July 2024)

This research note is restricted to the personal use of .

Table 3: Deployment Patterns and Their Use Cases, Benefits and Drawbacks

Deployment pattern	Use cases	Benefits	Drawbacks
Data services outside of Kubernetes	Data-managed services from infrastructure providers	Delegated management Integrated resilience features	Cost Potential for lock-in
Data services in separate Kubernetes cluster	Distributed databases Isolated and dedicated storage cluster Provides central storage for all applications	Isolates storage from application	Additional management overhead Potential bottleneck for other services if not well designed
Application includes data services	Test and dev Latency-sensitive applications	Close to application providing fast performance Fast provisioning	Less resilient Single point of failure

Source: Gartner (July 2024)

This research note is restricted to the personal use of .